

C  mputo Concurrente 2026

Pr  ctica 5

Snapshots y Collects: Backups

Profa: Gilde Valeria Rodr  guez

1 Contexto

Los snapshots at  micos son objetos concurrentes comunes, consisten de un arreglo de n   registros *MRSW*, los hilos pueden actualizar su propio registro (*update()*) o leer todos de forma at  mica o instant  nea (*scan()*).

Se han utilizado para simplificar el dise  o y la verificaci  n de algoritmos concurrentes de memoria compartida [AHR95]; por ejemplo, en el problema de *bounded timestamping* [GLS92], en general para construir objetos *wait-free* [ALS94] y para verificar algoritmos en tiempo de ejecuci  n [BFR  22, CnR23].

Los algoritmos concurrentes son dif  ciles de construir y verificar, existen diferentes t  cnicas para verificarlos. Una t  cnica muy famosa es la Verificaci  n formal, por ejemplo, se puede especificar un algoritmo concurrente utilizando lenguajes especializados como *TLA  * y *Coq*, de esta forma se comprueban todos los posibles estados del algoritmo [McC15]. Tambi  n existen t  cnicas m  s ligeras (utilizan menos c  mputo y son m  s f  ciles de aplicar) como la Verificaci  n en tiempo de ejecuci  n, consiste en obtener la ejecuci  n actual (*interfaz de comunicaci  n*) para decidir si la ejecuci  n es correcta o no hasta el momento (*interfaz de verificaci  n*).

En esta pr  ctica utilizaremos objetos snapshots at  micos y collects para construir una *interfaz de comunicaci  n* para verificar cualquier algoritmo concurrente en tiempo de ejecuci  n. El modelo para obtener la ejecuci  n de cualquier algoritmo es el siguiente:

Shared Variables:		
1:	snapshot <i>ssI</i> ;	�� Snapshot donde se guardan las invocaciones
2:	snapshot <i>ssR</i> ;	�� Snapshot donde se guardan las respuestas y vistas de las inv.
3:	alg. concurrente <i>A</i> ;	�� Algoritmo al cual aplicamos operaciones <i>op</i> (p. ej. <i>queue</i>)
4:	void run() {	
5:	<i>set</i> = conjunto de operaciones	
6:	<i>ssI.update(set �� op)</i>	�� Se guarda en <i>ssI</i> la sig. <i>op</i> a ejecutar
7:	<i>res</i> = <i>op</i>	�� Se ejecuta la operaci��n
8:	<i>view</i> = <i>ssI.scan()</i>	
9:	<i>ssR.update(view + res)</i>	�� Se guarda en <i>ssR</i> la vista y la respuesta de <i>op</i>
10:	}	

Figure 1: Modelo para verificar cualquier objeto *obj* de forma as  ncrona

Por ejemplo, en el c  digo (2) verificamos un algoritmo concurrente de una cola no bloqueante no acotada (). Se ejecut   el programa para 10 operaciones aleatorias de la cola, en la Figura a continuaci  n se observa lo que se obtiene, por cada hilo (de 0 a 3) se devuelven un conjunto de vistas, en cada vista se muestran las operaciones concurrentes o precedentes a la   ltima operaci  n del hilo. El hilo 0 primero hizo un *deg()*, devolvi   *null*, y solo hay una operaci  n precedente o concurrente a su operaci  n: *deg()* del hilo 1. Sabemos que las operaciones son concurrentes (*deg()* del hilo 0 y del hilo 1) porque el *deg()* del hilo 0 tambi  n est   en la primer vista correspondiente su *deg()*.

De hecho, podemos obtener la historia de la ejecuci  n, si seguimos las siguientes reglas (Figura 3):

Snapshot Final --- Values

Thread Owner: 0 Last Stamp: 4 Number of Snaps: 4
The values are:
[deq()] + [deq()] + [] + [] | null || deg():null
[deq(), enq(4)] + [deq(), deq()] + [enq(2)] + [enq(1)] | true || enq(4):true
[deq(), enq(4), deq()] + [deq(), deq()] + [enq(2), enq(7)] + [enq(1), deq()] | 4 || deg():4
[deq(), enq(4), deq(), deq()] + [deq(), deq()] + [enq(2), enq(7)] + [enq(1), deq()] | 7 || deg():7

Thread Owner: 1 Last Stamp: 2 Number of Snaps: 2
The values are:
[deq()] + [deq()] + [] + [] | null || deg():null
[deq(), enq(4), deq()] + [deq(), deq()] + [enq(2), enq(7)] + [enq(1), deq()] | 2 || deg():2

Thread Owner: 2 Last Stamp: 2 Number of Snaps: 2
The values are:
[deq(), enq(4)] + [deq(), deq()] + [enq(2)] + [enq(1)] | true || enq(2):true
[deq(), enq(4), deq()] + [deq(), deq()] + [enq(2), enq(7)] + [enq(1), deq()] | true || enq(7):true

Thread Owner: 3 Last Stamp: 2 Number of Snaps: 2
The values are:
[deq(), enq(4)] + [deq(), deq()] + [enq(2)] + [enq(1)] | true || enq(1):true
[deq(), enq(4), deq()] + [deq(), deq()] + [enq(2), enq(7)] + [enq(1), deq()] | 1 || deg():1

Figure 2

1. En cada vista existe una operación asociada op_a (subrayada en rojo), decimos que es *la vista de* op_a .
2. Una operación op_x en la vista de la operación op_a es **precedente** si op_a no está en la vista de op_x .
3. Una operación op_x en la vista de la operación op_a es **concurrente** si op_a está en la vista de op_x .

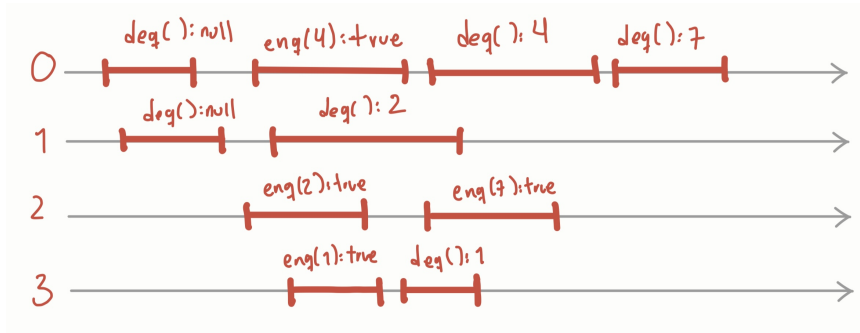


Figure 3

Finalmente, podemos verificar si esta historia es linealizable, existen algoritmos para verificar la linealizabilidad de un algoritmo concurrente, sin embargo, no ahondaremos en ellos. La idea es que cualquier hilo puede obtener la historia de la ejecución y verificarla de forma local.

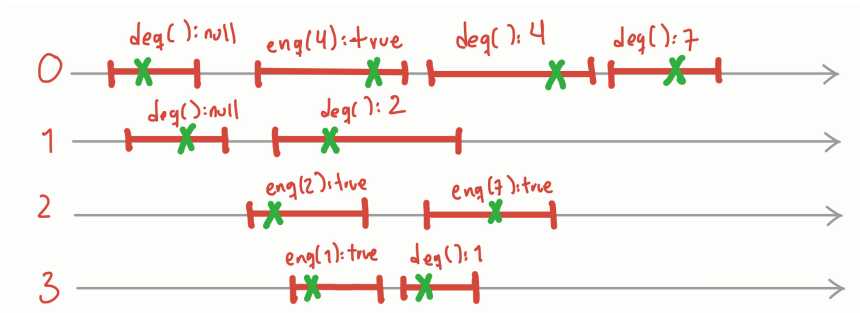


Figure 4

2 Ejemplos

En el siguiente link: https://github.com/surindt/FC_CConcurrente/tree/main/Programas_P5/unam.fc.concurrent.practica5/src/unam/fc/concurrent/practica5

1. Programa 1 (ExecuteSnapshot): Programa que ejecuta el *WFSnapshot*, imprime el contenido de los snaps actualizados por *update()* Utiliza la clase *WFSnapshot< T >*, la cual utiliza las clases *StampedSnap* y *StampedValue*.
2. Programa 2 (ExecuteSnapshotQ): Programa que ejecuta el *WFSnapshotH*, imprime el contenido de value del snapshot ssR. En el ssR se guardan las respuestas y las vistas, las vistas son un *scan()* del ssI de invocaciones. Utiliza la clase *WFSnapshotH< T >*, la cual utiliza *StampedSnapH* y *StampedValue*. El *WFSnapshotH* es diferente al *WFSnapshot* porque permite guardar todas las operaciones hechas en values y todos los snaps hechos en snap. Para ello se modifica *StampedSnapH* (a diferencia de *StampedSnap*) y se le agregaron listas para values y snap. Además utiliza la clase *RunnableQ < T >* para generar las operaciones aleatorias y realizar el modelo 1.
3. Programa 3 (SimpleSnapshot): Programa del snapshot obstruction-free

3 Ejercicios

- Entrega en un pdf las respuestas de los ejercicios que no requieran implementarse y añade una breve descripción de los programas que entregas (sus nombres y qué hacen). **El pdf como el directorio deben ir con el nombre y apellido paterno de la persona que entrega en Classroom (por ejemplo NombreApellido.pdf)**
- Debes realizar un programa en cada ejercicio que indique “Implementar”.
- El formato y el medio de entrega los indicará tu ayudante de laboratorio.
- Recuerda que debes utilizar Java 21 LTS.
- **Si no compila utilizando Java 21LTS o si no se entrega la breve descripción de los programas se penalizará.**

Tiempo de elaboración: $\approx$ 2hr

Total de puntos: 100

1. Implementa el mismo modelo que en el programa (2), reemplaza el Snapshot Wait-free por un Snapshot Obstruction-free (código en 3).
2. A partir de tu implementación, dibuja o describe una ejecución de 10 operaciones (como la de la Figura 3). ¿Observas alguna diferencia al utilizar un Snapshot obstruction-free en vez de uno wait-free? Argumenta porque crees que sucede así.
3. Implementa el mismo modelo que en el programa (2), reemplaza el Snapshot Wait-free por un Collect, para ello modifica el *scan()* del Snapshot obstruction-free (código en 3) como en la Tarea 4, en vez de hacer *double – collects* haz solo un *collect*.
4. Los collects, a diferencia de los snapshots, no son objetos atómicos, de hecho, ni siquiera tienen especificación secuencial. Sin embargo, analizando lo que se escribió en el collect, ¿crees que es posible obtener la ejecución así como con los snapshots? ¿Qué diferencias observas? Justifica tu respuesta
5. Investiga y escribe de forma breve en qué consiste el lenguaje *TLA+*, cuándo fue inventado y enumera 3 empresas que lo utilicen.

References

- [AHR95] Hagit Attiya, Maurice Herlihy, and Ophir Rachman. Atomic snapshots using lattice agreement. *Distributed Computing*, 13(9), 1995.
- [ALS94] Hagit Attiya, Nancy Lynch, and Nir Shavit. Are wait-free algorithms fast? *J. ACM*, 41(4):725–763, July 1994.
- [BFR⁺22] Borzoo Bonakdarpour, Pierre Frgaignaud, Sergio Rajsbaum, David Rosenblueth, and Corentin Travers. Decentralized asynchronous crash-resilient runtime verification. *J. ACM*, 69(5), October 2022.
- [CnR23] Armando Castañeda and Gilde Valeria Rodríguez. Asynchronous wait-free runtime verification and enforcement of linearizability. In *Proceedings of the 2023 ACM Symposium on Principles of Distributed Computing*, PODC ’23, page 90–101, New York, NY, USA, 2023. Association for Computing Machinery.
- [GLS92] Rainer Gawlick, Nancy Lynch, and Nir Shavit. Concurrent timestamping made simple. In *Symposium Proceedings on Theory of Computing and Systems*, ISTCS’92, page 171–183, Berlin, Heidelberg, 1992. Springer-Verlag.
- [McC15] Caitie McCaffrey. The verification of a distributed system: A practitioner’s guide to increasing confidence in system correctness. *Queue*, 13(9):150–160, December 2015.